

MANUAL TÉCNICO

MotosHub

Concesionario y taller de motocicletas en línea
React + Vite · FastAPI · MySQL · Inteligencia Artificial

Campo	Detalle
Programa de formación	Tecnólogo en Análisis y Desarrollo de Software (228118)
Ficha	3406211
Aprendiz	Tomás Cardona Hincapié
Instructor	César Augusto Moreno Mena
Proyecto	MotosHub — concesionario y taller de motocicletas
Repositorio	github.com/tomascardonahincapie/motoshub-react-fastapi
Aplicación publicada	motoshub-react-fastapi.vercel.app
Fecha	Septiembre de 2026

Tabla de contenido

1. Introducción y descripción general	4
1.1 Contexto y decisión de diseño sobre el pago	4
1.2 Actores del sistema	4
2. Objetivos	4
2.1 Objetivo general	4
2.2 Objetivos específicos	4
3. Alcance del proyecto	5
3.1 Funcionalidades incluidas	5
3.2 Funcionalidades excluidas explícitamente	5
4. Arquitectura de la solución	6
4.1 Stack tecnológico	6
4.2 Organización del Backend	7
5. Modelo de datos	8
5.1 Diccionario de datos	8
5.2 Decisiones del modelo que conviene explicar	9
6. Diseño de la solución	10
6.1 Historias de usuario principales	10
6.2 Flujo principal: del interés a la factura	10
7. Manual de instalación y configuración	11
7.1 Requisitos previos	11
7.2 Instalación del Backend	11
7.3 Instalación del Frontend	11
7.4 Variables de entorno	11
7.5 Despliegue en la nube	12
8. Documentación técnica de la API	13
8.1 Resumen por recurso	13
8.2 Diseño REST aplicado	14
8.3 Ejemplo de petición y respuesta	14
8.4 Seguridad de la API	15
9. Manual de usuario	16
9.1 Catálogo y pedido	16
9.2 Ingreso al sistema	16
9.3 Registro de una venta	17
9.4 Facturación	18
9.5 Dashboard e indicadores	18
9.6 PQR	19
9.7 Chatbot	20
10. Pruebas realizadas	21
10.1 Pruebas automáticas	21
10.2 Qué se comprueba, más allá del camino feliz	21
10.3 Pruebas de endpoints con Postman	22
10.4 Verificación en producción	22
11. Comparativa técnica: FastAPI frente a Django REST Framework	23
11.1 Por qué FastAPI en este proyecto	23
12. Conclusiones y recomendaciones	24

12.1 Conclusiones	24
12.2 Recomendaciones para una versión futura	24
13. Anexos	25
13.1 Enlaces del proyecto	25
13.2 Usuarios de prueba	25
13.3 Estructura del repositorio	25
13.4 Documentos adicionales	25

1. Introducción y descripción general

MotosHub es una aplicación web para un concesionario de motocicletas que además presta servicios de taller. Nace de un problema concreto del negocio: el catálogo vivía en redes sociales, los precios se repetían por mensajes sueltos y las ventas se anotaban a mano, de modo que al cierre del día nadie sabía con certeza cuánto se había vendido ni qué quedaba en inventario.

La aplicación reúne ese trabajo en un solo sitio: publica el catálogo con precios y disponibilidad reales, permite que el cliente arme su pedido y lo envíe por WhatsApp, y da al personal un panel donde registrar la venta ya acordada, emitir la factura, atender las peticiones, quejas y reclamos, y consultar indicadores.

1.1 Contexto y decisión de diseño sobre el pago

El negocio no cobra en línea. Una motocicleta no se compra por impulso: el cliente pregunta por financiación, traspaso, garantía y entrega antes de decidir. Por eso el sitio público no tiene pasarela de pago ni botón de compra: acompaña la decisión y encamina la conversación a WhatsApp, que es donde el negocio ya atiende.

La consecuencia técnica es importante y atraviesa todo el diseño: la venta se registra únicamente desde el panel, por un administrador o un empleado, cuando el trato ya está cerrado. Esa regla no se deja al criterio de la interfaz; el endpoint que crea ventas está reservado a esos dos roles y rechaza con 403 la petición de un cliente autenticado.

1.2 Actores del sistema

- Visitante: consulta el catálogo, arma un pedido, conversa con el chatbot y puede registrarse. No necesita cuenta para pedir por WhatsApp.
- Cliente: además de lo anterior, consulta sus compras, descarga sus facturas y radica PQR con número de radicado.
- Empleado: registra ventas ya acordadas, emite facturas, responde PQR y consulta el reporte del día.
- Administrador: todo lo anterior, más la gestión de usuarios, del catálogo y el acceso a los indicadores completos.

2. Objetivos

2.1 Objetivo general

Construir y poner en producción una aplicación web completa para un concesionario y taller de motocicletas, que publique el catálogo, encamine los pedidos por WhatsApp, registre las ventas acordadas con su facturación e inventario, atienda las PQR y entregue indicadores de gestión, sobre una API REST propia con FastAPI y una base de datos relacional.

2.2 Objetivos específicos

- Diseñar un modelo relacional normalizado que sostenga el ciclo comercial completo: catálogo, venta, detalle, factura y atención al cliente, con la integridad garantizada por claves foráneas en la propia base.
- Implementar una API REST con FastAPI que separe routers, esquemas, acceso a datos y servicios, y que valide la entrada con Pydantic v2 antes de tocar la base.
- Proteger la aplicación con autenticación JWT y control de acceso por rol verificado en el servidor, de modo que ocultar un botón nunca sea la única barrera.
- Desarrollar una interfaz en React que consuma la API mediante un cliente HTTP centralizado y maneje los estados de carga y de error.

- Integrar un servicio de Inteligencia Artificial para el chatbot de atención, alimentado con el catálogo real y con la clave gestionada por variables de entorno.
- Desplegar la solución en la nube y verificar su funcionamiento contra los dominios públicos.

3. Alcance del proyecto

3.1 Funcionalidades incluidas

Módulo	Qué resuelve
Catálogo	12 motocicletas y 8 servicios de taller, con búsqueda, filtro por categoría, orden y ficha de detalle
Pedido por WhatsApp	El visitante arma su lista, ve el total de referencia y la envía con los artículos ya redactados
Ventas	Registro desde el panel con descuento de inventario, IVA y consecutivo propio
Facturación	Emisión con su propio detalle, consulta, cambio de estado y descarga en PDF
Reportes	Reporte diario de ventas exportable a PDF y a Excel con fórmulas vivas
Dashboards	Indicadores, gráfico de barras, tendencia lineal, ranking y repartos, recortados según el rol
PQR	Radicación con número, seguimiento de estado y respuesta desde el panel
Chatbot	Atención con IA alimentada con el catálogo real, con motor de reglas como alternativa
Usuarios y roles	Registro, autenticación JWT, recuperación de contraseña y gestión de tres roles

3.2 Funcionalidades excluidas explícitamente

Se dejan fuera del alcance, por decisión de negocio o por no ser verificables en este entorno:

- Pago en línea y pasarela de pagos. No hay cobro por la web: toda forma de pago se acuerda por WhatsApp.
- Integración automática con la API de WhatsApp Business. El sitio abre la conversación con el mensaje redactado, pero la aplicación no puede saber qué se responde allí; por eso la venta la registra una persona.
- Facturación electrónica ante la DIAN. La factura que emite el sistema es un comprobante interno, no un documento tributario validado.
- Aplicación móvil nativa. La interfaz es responsiva, pero no hay app de tienda.
- Gestión de proveedores, compras a fábrica y contabilidad.

4. Arquitectura de la solución

La aplicación se organiza en tres capas: una interfaz en React que corre en el navegador, una API REST en FastAPI que concentra todas las reglas de negocio, y una base de datos MySQL. Cada capa vive en un proveedor distinto y se comunica por HTTPS.

La decisión de fondo es que el navegador no decide nada: pide y muestra. Los precios se leen del catálogo en el servidor aunque la petición traiga otros, el alcance de cada consulta se recorta según el rol que viaja en el token, y las reglas de negocio viven en la capa de datos, no en la pantalla.

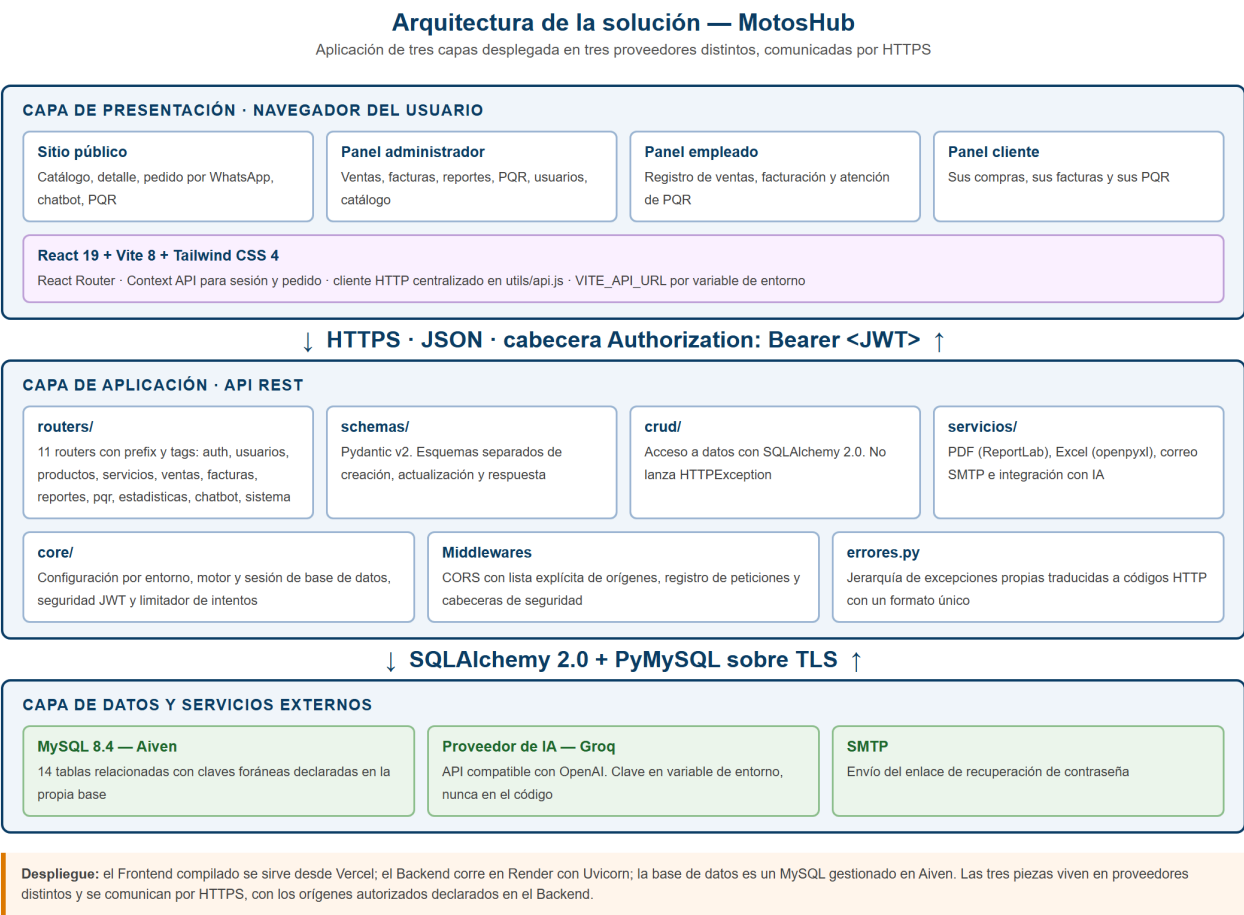


Figura 1. Arquitectura en tres capas y despliegue en la nube.

4.1 Stack tecnológico

Capa	Tecnología	Versión	Para qué
Frontend	React	19.2	Interfaz de usuario por componentes
Frontend	Vite	8	Servidor de desarrollo y compilación
Frontend	Tailwind CSS	4	Sistema de estilos con tokens propios
Frontend	React Router	7	Rutas del lado del navegador
Backend	FastAPI	0.141.1	API REST y documentación automática
Backend	Uvicorn	0.52.4	Servidor ASGI
Backend	SQLAlchemy	2.0.52	ORM con Mapped y mapped_column
Backend	Pydantic	2.13.5	Validación de entrada y salida
Backend	PyMySQL	1.2.0	Controlador de MySQL

Backend	python-jose / bcrypt	3.5.0 / 5.0.0	Firma de JWT y hash de contraseñas
Backend	ReportLab / openpyxl	4.4.4 / 3.1.5	Facturas y reportes en PDF y Excel
Datos	MySQL	8.4	Base de datos relacional
Pruebas	Pytest / httpx	9.1.1 / 0.28.1	Pruebas automáticas sobre TestClient
IA	Groq (API compatible OpenAI)	gpt-oss-120b	Motor del chatbot

4.2 Organización del Backend

El código del Backend se reparte por responsabilidad, no por recurso, de modo que cada archivo tenga una sola razón para cambiar:

Carpeta	Contenido
app/routers/	Un APIRouter por recurso, con su prefix y sus tags. Solo traduce peticiones y respuestas
app/schemas/	Esquemas Pydantic separados para crear, actualizar y responder
app/crud/	Acceso a datos con SQLAlchemy. No conoce HTTP ni lanza HTTPException
app/models/	Modelos ORM con DeclarativeBase, Mapped y mapped_column
app/core/	Configuración por entorno, motor y sesión de base de datos, seguridad y limitador de intentos
app/servicios/	PDF, Excel, correo e integración con el proveedor de IA
app/errores.py	Jerarquía de excepciones propias del dominio
app/main.py	Solo configuración de la app, middlewares y include_router

5. Modelo de datos

La base tiene 14 tablas relacionadas. Las claves foráneas están declaradas en la propia base y no solo en el ORM: si alguien entra por consola y borra un usuario con ventas, la base lo impide.

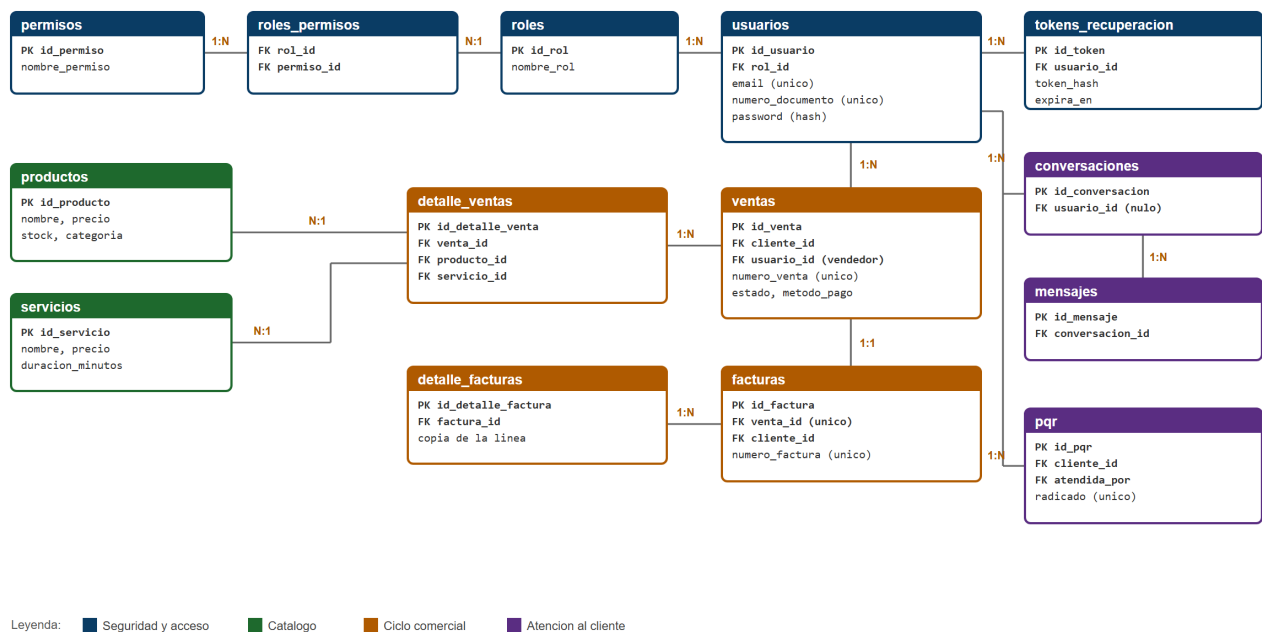


Figura 2. Diagrama entidad-relación de las 14 entidades.

5.1 Diccionario de datos

Se listan las entidades con su propósito, sus campos principales y sus relaciones. El detalle completo de tipos y restricciones está en backend/database/schema.sql.

Entidad	Propósito	Campos principales	Relaciones
roles	Los tres perfiles del sistema	id_rol (PK), nombre_rol	1:N con usuarios
permisos	Acciones que un rol puede ejecutar	id_permiso (PK), nombre_permiso	N:M con roles
roles_permisos	Tabla puente de la relación N:M	rol_id (FK), permiso_id (FK)	N:1 con roles y permisos
usuarios	Personas que usan el sistema	id_usuario (PK), rol_id (FK), email (único), numero_documento (único), password (hash bcrypt)	1:N con ventas, facturas, pqr, conversaciones
tokens_recuperacion	Enlaces de recuperación de contraseña	id_token (PK), usuario_id (FK), token_hash, expira_en, usado_en	N:1 con usuarios
productos	Catálogo de motocicletas	id_producto (PK), nombre, descripcion, precio, stock, categoria, imagen	1:N con detalle_ventas
servicios	Catálogo del taller	id_servicio (PK), nombre, precio, duracion_minutos, imagen	1:N con detalle_ventas
ventas	Cabecera de cada operación comercial	id_venta (PK), numero_venta (único), cliente_id (FK), usuario_id (FK, vendedor), subtotal, descuento, impuestos, total, estado, metodo_pago, fecha_venta	1:N con detalle_ventas, 1:1 con facturas

detalle_ventas	Una línea por artículo vendido	id_detalle_venta (PK), venta_id (FK), producto_id (FK), servicio_id (FK), nombre_item, cantidad, precio_unitario, subtotal	N:1 con ventas, productos y servicios
facturas	Comprobante emitido de una venta	id_factura (PK), numero_factura (único), venta_id (FK, único), cliente_id (FK), cliente_nombre, cliente_documento, porcentaje_iva, total, estado, fecha_emision	1:1 con ventas, 1:N con detalle_facturas
detalle_facturas	Copia de las líneas al momento de facturar	id_detalle_factura (PK), factura_id (FK), nombre_item, cantidad, precio_unitario, subtotal	N:1 con facturas
pqr	Peticiones, quejas y reclamos	id_pqr (PK), radicado (único), cliente_id (FK), atendida_por (FK), tipo, asunto, descripcion, estado, respuesta	N:1 con usuarios
conversaciones	Hilo de chat con el asistente	id_conversacion (PK), usuario_id (FK, admite nulo)	1:N con mensajes
mensajes	Cada turno de la conversación	id_mensaje (PK), conversacion_id (FK), autor, contenido, origen	N:1 con conversaciones

5.2 Decisiones del modelo que conviene explicar

El detalle de la venta guarda una copia del nombre y del precio del artículo. Si mañana una moto cambia de precio o sale del catálogo, el histórico no se reescribe solo.

La factura, a su vez, guarda su propia copia de las líneas en detalle_facturas, además de los datos del cliente y los totales. Así, corregir el detalle de una venta ya facturada no altera el documento emitido, que es como se comporta una factura de verdad.

conversaciones.usuario_id admite nulo a propósito: un visitante sin cuenta puede conversar con el chatbot, y esa conversación se guarda igual.

6. Diseño de la solución

6.1 Historias de usuario principales

ID	Historia de usuario	Funcionalidad que la implementa
HU-01	Como visitante quiero ver las motos disponibles con su precio y existencias para decidir sin preguntar	Catálogo público con filtros, orden y ficha de detalle
HU-02	Como visitante quiero enviar mi pedido por WhatsApp sin crear una cuenta	Panel de pedido que arma el mensaje con artículos y total
HU-03	Como visitante quiero preguntar a cualquier hora qué moto me conviene	Chatbot con IA alimentado con el catálogo real
HU-04	Como empleado quiero registrar la venta ya acordada y que el inventario se descuente solo	POST /api/ventas con descuento de stock en la misma transacción
HU-05	Como empleado quiero emitir la factura y entregarla en PDF	POST /api/facturas y GET /api/facturas/{id}/pdf
HU-06	Como administrador quiero saber al cierre cuánto se vendió hoy	Reporte diario exportable a PDF y Excel
HU-07	Como administrador quiero ver la tendencia y lo más vendido	Dashboard con serie, ranking y repartos
HU-08	Como cliente quiero consultar mis compras y descargar mis facturas	Panel del cliente, con el alcance recortado en el servidor
HU-09	Como cliente quiero radicar un reclamo y hacerle seguimiento	Módulo de PQR con número de radicado y estados
HU-10	Como administrador quiero que un cliente no pueda ver ni tocar lo ajeno	Dependencias de rol y recorte por alcance en cada consulta

6.2 Flujo principal: del interés a la factura

1. El visitante encuentra la moto en el catálogo y la añade a su pedido, o pregunta por ella al chatbot.
2. Envía el pedido por WhatsApp. El mensaje llega con los artículos, las cantidades y el total de referencia.
3. El negocio responde por WhatsApp: confirma disponibilidad, acuerda la forma de pago y la entrega.
4. Con el trato cerrado, el empleado registra la venta desde el panel, eligiendo el cliente y los artículos. FastAPI toma los precios del catálogo, calcula el IVA, descuenta el inventario y devuelve el consecutivo.
5. La factura se emite en el mismo momento, con su propia copia de las líneas, y queda disponible en PDF tanto para el panel como para el cliente.
6. La venta entra de inmediato en el reporte del día y en los indicadores del Dashboard.

7. Manual de instalación y configuración

7.1 Requisitos previos

- Python 3.12 o superior
- Node.js 20 o superior
- MySQL 8 (en desarrollo se usó el de XAMPP)
- Git

7.2 Instalación del Backend

Desde la raíz del repositorio:

Paso	Comando
1. Entrar a la carpeta	cd backend
2. Crear el entorno virtual	python -m venv venv
3. Activarlo (Windows)	venvScriptsactivate
4. Instalar dependencias	pip install -r requirements.txt
5. Copiar la plantilla de entorno	copy .env.example .env
6. Cargar la base de datos	python scripts/cargar_esquema.py
7. Generar datos de demostración	python scripts/datos_demo.py
8. Levantar la API	uvicorn app.main:app --reload

La API queda en `http://127.0.0.1:8000` y su documentación en `/docs`.

7.3 Instalación del Frontend

Paso	Comando
1. Entrar a la carpeta	cd frontend
2. Instalar dependencias	npm install
3. Levantar el servidor	npm run dev

El sitio queda en `http://localhost:5173`.

7.4 Variables de entorno

Ninguna credencial vive en el código. Todas se leen del entorno mediante `pydantic-settings`; el archivo `.env` está en `.gitignore` y en el repositorio solo viaja `.env.example`, sin valores reales.

Variable	Para qué sirve	Ejemplo
DATABASE_URL	Conexión completa que entrega la plataforma. Si tiene valor, manda sobre las DB_*	mysql://usuario:clave@host:puerto/base
DB_HOST, DB_USER, DB_PASSWORD, DB_NAME	Conexión por partes, para desarrollo local	localhost / root / (vacío) / bd_jhm_tech_solutions
JWT_SECRET	Clave con que se firman los tokens. Sin valor por defecto utilizable en producción	cadena larga y aleatoria
JWT_EXPIRA_MINUTOS	Vigencia del token	480

ENTORNO	desarrollo o produccion	produccion
DEPURACION	Silencia los mensajes de error detallados	false
ORIGENES_PERMITIDOS	Lista explícita de orígenes autorizados para CORS. Nunca comodín	https://mi-sitio.vercel.app
URL_FRONTEND	Base del enlace de recuperación de contraseña	https://mi-sitio.vercel.app
IA_PROVEEDOR / IA_URL_BASE / IA_MODELO / IA_API_KEY	Integración del chatbot con el proveedor de IA	openai / https://api.groq.com/openai/v1 / openai/gpt-oss-120b
SMTP_HOST, SMTP_USUARIO, SMTP_PASSWORD	Envío del correo de recuperación. Si se dejan vacíos, el enlace se escribe en consola	smtp.gmail.com
IVA_PORCENTAJE	Porcentaje aplicado a la base gravable	19

En el Frontend la única variable es VITE_API_URL, que apunta a la API terminada en /api. Se lee al compilar, no al arrancar: si cambia, hay que volver a desplegar el Frontend.

7.5 Despliegue en la nube

La aplicación está publicada en tres proveedores. El procedimiento completo está en DESPLIEGUE.md.

Pieza	Proveedor	Configuración
Frontend	Vercel	Root Directory frontend, variable VITE_API_URL
Backend	Render	Root Directory backend, comando uvicorn app.main:app --host 0.0.0.0 --port \$PORT, health check en /salud
Base de datos	Aiven	MySQL 8.4 gestionado, conexión por TLS

Dos advertencias aprendidas en el despliegue real. La primera: el esquema crea la base con su propio nombre, mientras que la plataforma entrega una base ya creada con otro; si no se ajusta el nombre en DATABASE_URL, el servicio arranca y no encuentra nada. La segunda: sin declarar el dominio del Frontend en ORIGENES_PERMITIDOS, el navegador bloquea todas las peticiones por CORS y el catálogo aparece vacío sin mensaje de error.

8. Documentación técnica de la API

La API expone 45 operaciones repartidas en 32 rutas y 11 etiquetas. Cada endpoint declara su tag, su summary y los códigos de error que puede devolver, de modo que la documentación automática en /docs sea utilizable como referencia y no solo como lista.

Ventas Registro de ventas e historial con filtros.			^
POST	/api/ventas	Registrar una venta	🔒 ▼
GET	/api/ventas	Consultar el historial de ventas	🔒 ▼
GET	/api/ventas/{id_venta}	Consultar una venta con su detalle	🔒 ▼
PATCH	/api/ventas/{id_venta}/estado	Cambiar el estado de una venta	🔒 ▼
Facturas Emisión, consulta y descarga de facturas en PDF.			^
POST	/api/facturas	Emitir la factura de una venta	🔒 ▼
GET	/api/facturas	Consultar facturas	🔒 ▼
GET	/api/facturas/{id_factura}	Consultar una factura	🔒 ▼
GET	/api/facturas/{id_factura}/pdf	Descargar la factura en PDF	🔒 ▼
PATCH	/api/facturas/{id_factura}/estado	Cambiar el estado de una factura	🔒 ▼
Reportes Reporte diario de ventas en JSON, PDF y Excel.			^
GET	/api/reportes/ventas-diarias	Reporte diario de ventas	🔒 ▼
GET	/api/reportes/ventas-diarias/pdf	Descargar el reporte diario en PDF	🔒 ▼
GET	/api/reportes/ventas-diarias/excel	Descargar el reporte diario en Excel	🔒 ▼
PQR Peticiones, quejas, reclamos y sugerencias.			^
POST	/api/pqr	Radicar una PQR	🔒 ▼

Figura 3. Documentación automática en /docs, agrupada por etiquetas.

8.1 Resumen por recurso

Etiqueta	Operaciones	Qué agrupa
Autenticacion	5	Registro, inicio de sesión y recuperación de contraseña
Usuarios	6	CRUD completo y cambio de estado
Productos	5	CRUD del catálogo de motocicletas
Servicios	5	CRUD del catálogo del taller
Ventas	4	Registro, historial, detalle y cambio de estado
Facturas	5	Emisión, consulta, estado y descarga en PDF
Reportes	3	Reporte diario en pantalla, PDF y Excel
PQR	4	Radicación, consulta y respuesta
Estadísticas	2	Indicadores y serie del Dashboard
Chatbot	3	Conversación, estado de la IA e historial
Sistema	2	Información del servicio y estado de salud

8.2 Diseño REST aplicado

Las rutas nombran recursos en plural y no contienen verbos. Las operaciones de negocio que no son un simple cambio de campo se modelan como sub-recurso, por ejemplo PATCH `/api/ventas/{id}/estado` en lugar de un PATCH genérico sobre la venta.

Recurso	Verbo	Ruta	Código
Venta	POST	/api/ventas	201
Venta	GET	/api/ventas	200
Venta	GET	/api/ventas/{id_venta}	200 / 404
Venta	PATCH	/api/ventas/{id_venta}/estado	200 / 409
Factura	POST	/api/facturas	201 / 409
Factura	GET	/api/facturas/{id_factura}/pdf	200 / 403
Producto	POST	/api/productos	201
Producto	PUT	/api/productos/{id_producto}	200 / 404
Producto	DELETE	/api/productos/{id_producto}	204
PQR	POST	/api/pqr	201
PQR	PATCH	/api/pqr/{id_pqr}	200 / 403

El cliente nunca envía los campos que decide el servidor. El número de venta, la fecha, el estado y los totales se calculan en el Backend; si la petición los trae, se ignoran. Lo mismo con el precio: se lee del catálogo, y hay una prueba automática que lo comprueba enviando un precio falso.

8.3 Ejemplo de petición y respuesta

Registro de una venta. Requiere token de administrador o empleado; un cliente recibe 403.

```
POST /api/ventas
Authorization: Bearer <token>
Content-Type: application/json

{
  "cliente_id": 3,
  "metodo_pago": "efectivo",
  "generar_factura": true,
  "items": [
    { "tipo_item": "producto", "id_item": 1, "cantidad": 1 },
    { "tipo_item": "servicio", "id_item": 2, "cantidad": 1 }
  ]
}
```

Respuesta 201 Created:

```
{
  "ok": true,
  "message": "Venta V-2026-000060 registrada correctamente",
  "id_venta": 60,
  "numero_venta": "V-2026-000060",
  "total": "30553500.00",
  "id_factura": 60,
  "numero_factura": "FV-2026-000060"
}
```

Todos los errores salen con el mismo formato, de modo que el Frontend los trata igual venga de donde venga:

```
{
  "ok": false,
  "message": "No tienes permisos para acceder a este recurso",
  "codigo": "sin_permisos"
}
```

8.4 Seguridad de la API

- Las contraseñas se guardan con bcrypt y nunca se devuelven en ninguna respuesta.
- El login entrega un JWT firmado que lleva el identificador del usuario, su rol y la expiración.
- Una dependencia valida el token en cada petición protegida y entrega el usuario ya resuelto.
- Se distingue 401 (no hay sesión) de 403 (hay sesión pero no alcanza el rol).
- El alcance de las consultas se recorta en el servidor: un cliente que pida el historial recibe solo sus compras, aunque fuerce los parámetros.
- CORS se configura con una lista explícita de orígenes tomada del entorno; no se usa comodín.
- La recuperación de contraseña tiene tres topes: por correo, por IP y un tiempo mínimo entre envíos, y responde siempre lo mismo exista o no la cuenta, para no delatar qué direcciones están registradas.

9. Manual de usuario

Esta sección recorre la aplicación desde el punto de vista de quien la usa. Las capturas están tomadas del sistema en funcionamiento contra la base de datos real.

9.1 Catálogo y pedido

El catálogo muestra las motocicletas con su foto, precio y unidades disponibles, con buscador, filtro por categoría y ordenamiento. Cada tarjeta ofrece tres caminos: ver la ficha completa, añadir la moto al pedido, o preguntar por ella directamente por WhatsApp.



Figura 4. Catálogo público. No hay botón de compra: se añade al pedido o se pregunta por WhatsApp.

El panel lateral del pedido muestra el subtotal, el IVA y el total de referencia, y lo envía por WhatsApp con los artículos ya redactados. No se cobra nada en la web:

```
Hola MotosHub, me interesa este pedido:

- Honda CB 125F x1 - $ 8.900.000

Total aproximado: $ 10.591.000 (IVA incluido)

Me confirman disponibilidad y como seguimos?
```

9.2 Ingreso al sistema

El inicio de sesión va en dos pasos, como el de los servicios grandes: primero el correo y después la contraseña, con la cuenta elegida a la vista y un botón para corregirla. El paso entre ambos no consulta al servidor a propósito: si preguntara ahí si el correo existe, cualquiera podría averiguar qué direcciones tienen cuenta. El mensaje de fallo es el mismo sin distinguir si sobraba el correo o la contraseña.



INICIOCATÁLOGONOSOTROSCONTACTO

INICIAR SESIÓN



BIENVENIDO DE VUELTA A MOTOSHUB

Entra con tu cuenta para acceder a tu panel y gestionar tus compras y servicios de taller.

- ✓ Compra tu moto en línea, sin filas
- ✓ Agenda servicios del taller por WhatsApp
- ✓ Consulta el estado de tu cuenta cuando quieras

INICIAR SESIÓN

Empecemos por tu correo electrónico

CORREO ELECTRÓNICO

lucorreo@ejemplo.com0/100

SIGUIENTE

¿No tienes cuenta? [Crear cuenta](#)

¿Solo quieres mirar? [Ver el catálogo](#)



Venta de motocicletas nuevas, con taller propio para el mantenimiento de tu moto.

ESCRÍBENOS

NAVIGACIÓN

- Inicio
- Catálogo
- Nosotros
- Contacto

TU CUENTA

- Iniciar sesión
- Productos
- Servicios de taller

CONTACTO

 Cra. 45 #12-30, Medellín, Colombia

 contacto@motoshub.com

 +57 300 000 0000

 Lun a Vie 8:00 - 18:00 · Sáb 9:00 - 14:00



Figura 5. Primer paso del ingreso: solo el correo.

9.3 Registro de una venta

Con el trato ya cerrado por WhatsApp, el empleado o el administrador registra la venta desde el panel: elige el cliente, la forma de pago acordada y los artículos. Los precios los pone el servidor leyendo el catálogo.



ADMINISTRACIÓN

Dashboard

Indicadores y gráficos de toda la operación

REGISTRAR VENTA

  Admin ADMINISTRADOR

Dashboard

Ventas

Facturas

Reportes

PQR

Usuarios

Motos

Servicios

Registrar una venta

Los precios se toman del catálogo al confirmar

CLIENTE

Selecciona el cliente

FORMA DE PAGO

Efectivo

ARTÍCULO

Elige un producto o servicio

CANTIDAD

1

AÑADIR

OBSERVACIONES (OPCIONAL)

Entrega en tienda, garantía extendida...

Figura 6. Formulario de registro de venta en el panel.

9.4 Facturación

La factura se emite junto con la venta o después, y queda disponible para consulta, cambio de estado y descarga en PDF. El cliente ve únicamente las suyas.

FACTURA	EMISIÓN	CLIENTE	DOCUMENTO	ESTADO	TOTAL	
FV-2026-000055 V-2026-000055	18 sep 2026 · 14:00	jostin cebolla	1025654103	PAGADA	\$ 160.650	PDF
FV-2026-000054 V-2026-000054	18 sep 2026 · 10:45	jostin cebolla	1025654103	EMITIDA	\$ 142.800	MARCAR PAGADA PDF
FV-2026-000051 V-2026-000051	17 sep 2026 · 16:30	jostin cebolla	1025654103	EMITIDA	\$ 59.500	MARCAR PAGADA PDF
FV-2026-000053 V-2026-000053	17 sep 2026 · 10:15	cliente algo	1025654106	EMITIDA	\$ 166.600	MARCAR PAGADA PDF
FV-2026-000052 V-2026-000052	17 sep 2026 · 10:00	Carlos Perez	1000000002	PAGADA	\$ 166.600	PDF
FV-2026-000050 V-2026-000050	16 sep 2026 · 17:30	cliente algo	1025654106	PAGADA	\$ 95.200	PDF
FV-2026-000049 V-2026-000049	16 sep 2026 · 13:15	jostin cebolla	1025654103	ANULADA	\$ 95.200	PDF
FV-2026-000048 V-2026-000048	16 sep 2026 · 09:15	Carlos Perez	1000000002	PAGADA	\$ 142.800	PDF
FV-2026-000046 V-2026-000046	12 sep 2026 · 17:30	cliente algo	1025654106	PAGADA	\$ 154.700	PDF
FV-2026-000045 V-2026-000045	12 sep 2026 · 13:30	jostin cebolla	1025654103	PAGADA	\$ 202.300	PDF

Figura 7. Consulta de facturas con búsqueda y filtros.

9.5 Dashboard e indicadores

El Dashboard reúne las tarjetas de indicadores, el gráfico de barras por periodo, la tendencia lineal, el ranking de lo más vendido y el reparto por estado y forma de pago. Lo que cada rol ve lo decide el Backend, no la pantalla: el cliente recibe solo sus propias cifras.

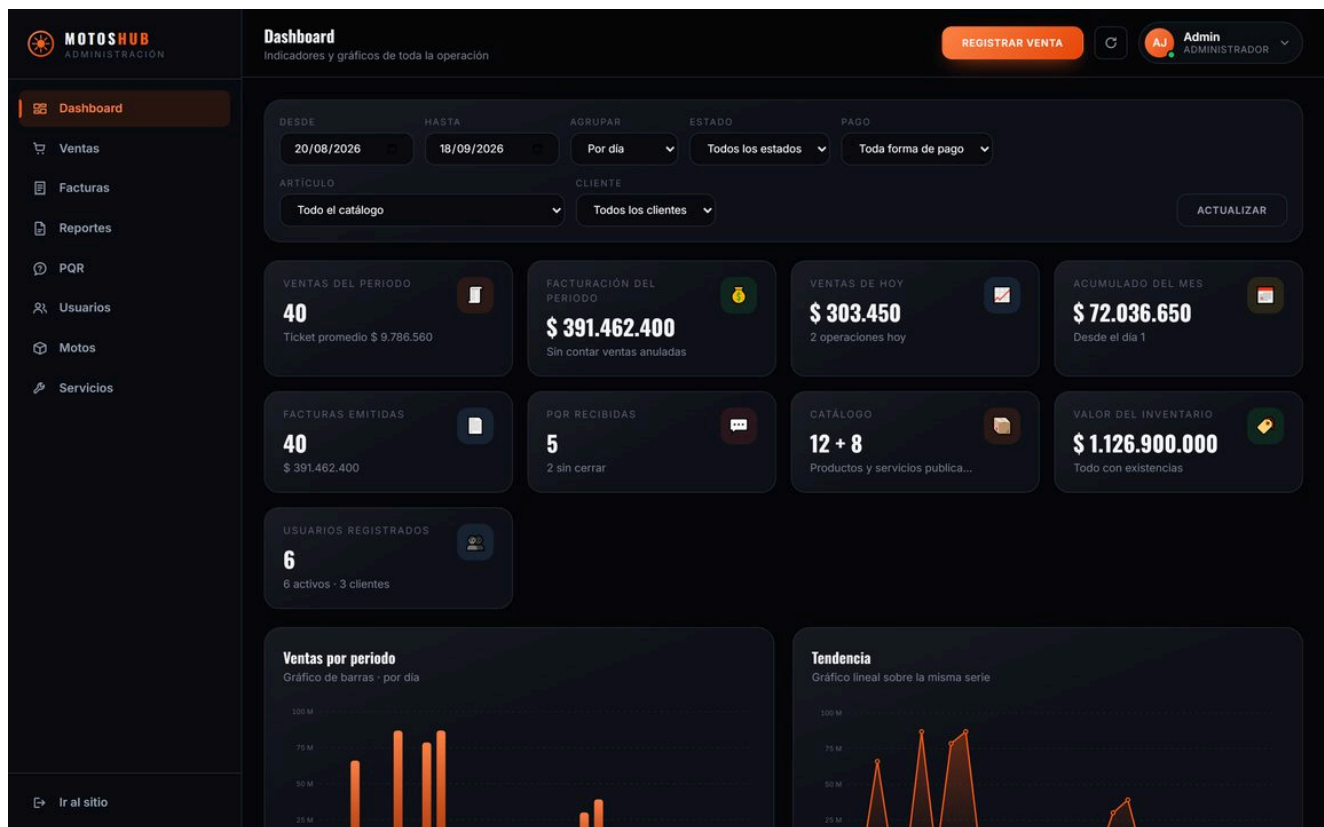


Figura 8. Dashboard administrativo con filtros por fecha, estado y forma de pago.

9.6 PQR

El cliente radica su petición, queja o reclamo y recibe un número de radicado con el que sigue su estado. Desde el panel se responde y se cambia el estado entre pendiente, en proceso, respondida y cerrada.

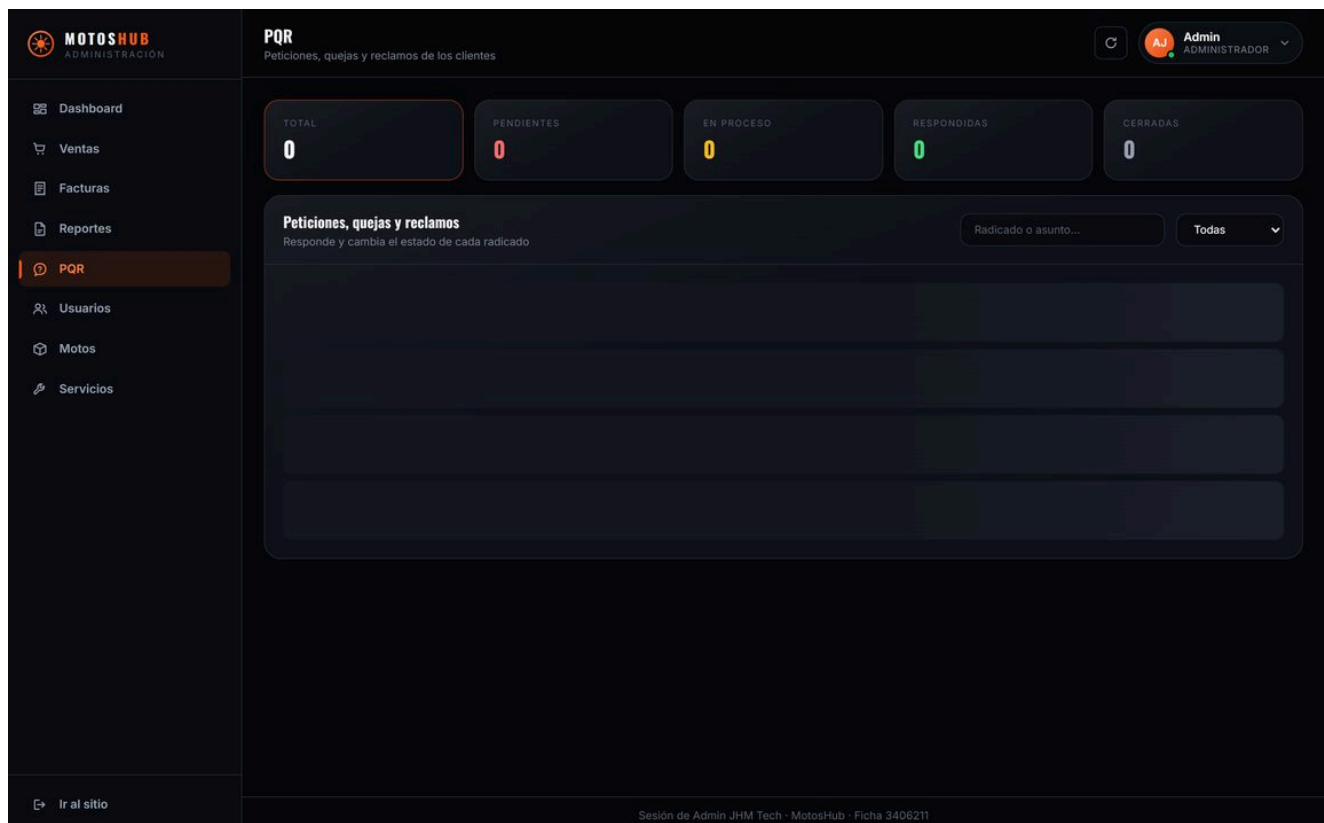


Figura 9. Gestión y respuesta de una PQR desde el panel.

9.7 Chatbot

El asistente atiende en el sitio público, con o sin cuenta. Responde con Inteligencia Artificial alimentada con el catálogo real, de modo que los precios y las existencias que cita salen de la base de datos. Si el proveedor falla o no hay clave configurada, el chatbot no se cae: pasa a un motor de reglas que consulta esa misma base y avisa de la situación.



Figura 10. Conversación con el asistente en el sitio público.

10. Pruebas realizadas

10.1 Pruebas automáticas

El proyecto tiene 137 pruebas automáticas escritas con Pytest sobre el TestClient de FastAPI. Corren contra una base de datos aislada, distinta de la de desarrollo, que se crea y se destruye en cada ejecución mediante `dependency_overrides`. Los servicios externos —el proveedor de IA y el correo— se sustituyen por dobles, de modo que la suite no depende de la red ni gasta cuota.

Archivo	Qué cubre
test_api.py	Registro, inicio de sesión, JWT, roles y CRUD de usuarios, productos y servicios
test_ventas.py	Registro de ventas, precios desde el catálogo, inventario, permisos y anulación
test_facturas_reportes.py	Emisión, copia del detalle, descarga en PDF y reportes diarios
test_pqr_dashboard.py	Radicación y respuesta de PQR, e indicadores según el rol
test_chatbot.py	Conversación con IA simulada, motor de reglas y protección de la clave
test_recuperacion.py	Enlace de recuperación, expiración y límites de envío
test_configuracion.py	Lectura de la configuración del entorno y armado de la cadena de conexión

```
MOTOSHUB - QUINTO AVANCE
118 pruebas automáticas en verde
Cubren registro, inicio de sesión, JWT, roles, CRUD, ventas, stock, facturación, reportes en PDF y Excel, PQR, Dashboards y chatbot.

BACKEND/TESTS/
$ ls backend/tests/
__init__.py
conf_test.py
test_api.py
test_chatbot.py
test_configuracion.py
test_facturas_reportes.py
test_pqr_dashboard.py
test_recuperacion.py
test_ventas.py

PYTEST
$ pytest -q

plugins: anyio-4.15.1
collected 133 items
tests/test_api.py ..... [ 30%]
tests/test_chatbot.py ..... [ 42%]
tests/test_configuracion.py ..... [ 46%]
tests/test_facturas_reportes.py ..... [ 59%]
tests/test_pqr_dashboard.py ..... [ 71%]
tests/test_recuperacion.py ..... [ 86%]
tests/test_ventas.py ..... [100%]

===== 133 passed in 188.84s (0:03:08) =====
```

Figura 11. Ejecución de la suite completa.

10.2 Qué se comprueba, más allá del camino feliz

Las pruebas no se limitan a confirmar que lo que debe funcionar funciona. Cubren también cada error de negocio del dominio:

- Enviar un precio falso en la petición no abarata la compra: el total se calcula con el precio del catálogo.

- Vender más unidades de las que hay en inventario se rechaza, y varias líneas del mismo producto se suman antes de mirar el stock.
- Un cliente no puede registrar ventas, ni ver las de otro, ni anular la suya, ni forzar los filtros para espiar el historial ajeno.
- Una venta no se puede facturar dos veces; el segundo intento responde 409.
- Corregir el detalle de una venta ya facturada no altera la factura emitida.
- Una venta anulada devuelve las unidades al inventario y deja de sumar en el historial.
- Si el proveedor de IA falla, el chatbot sigue atendiendo con el motor de reglas.
- El endpoint de estado del chatbot nunca devuelve la clave, ni siquiera parcialmente.

10.3 Pruebas de endpoints con Postman

La colección backend/postman/MotosHub_API.postman_collection.json reúne 86 peticiones en 13 carpetas, con 97 comprobaciones automáticas que ejercitan los cinco verbos: 45 GET, 26 POST, 8 PATCH, 4 DELETE y 3 PUT. La colección resuelve los identificadores consultando el catálogo vivo en lugar de traerlos escritos a mano, de modo que puede ejecutarse repetidamente sin dejar rastro. Se corre sin abrir Postman con:

```
npx newman run backend/postman/MotosHub_API.postman_collection.json
```

10.4 Verificación en producción

Además de las pruebas locales, se verificó la aplicación contra los dominios públicos: /salud respondiendo con la base conectada, el catálogo sirviendo las 12 motocicletas, el inicio de sesión entregando token, los indicadores del Dashboard, la descarga del reporte en PDF y en Excel, el chatbot respondiendo con IA y el preflight de CORS autorizando el dominio del Frontend.

11. Comparativa técnica: FastAPI frente a Django REST Framework

La elección del framework no fue por moda. Se contrastaron los dos candidatos habituales en Python para una API REST, aplicados a lo que este proyecto necesitaba.

Aspecto	FastAPI (elegido)	Django REST Framework
Validación	Pydantic v2 sobre las anotaciones de tipo. El mismo tipo sirve para validar y para documentar	Serializers propios. Potentes, pero la validación y el tipado van por separado
Documentación	OpenAPI y Swagger generados del código, sin escribir nada aparte	Requiere drf-spectacular o drf-yasg como añadido
ORM	Libre. Aquí SQLAlchemy 2.0, que permite modelar sin atarse al framework	El ORM de Django viene incorporado y es difícil de sustituir
Estructura	La define el equipo. Exige criterio, pero permitió separar routers, schemas, crud y servicios como convenía	Convención por aplicaciones. Más guiada, menos flexible
Asincronía	Nativa desde el diseño, sobre Starlette y ASGI	Soporte añadido con los años; buena parte del ecosistema sigue siendo síncrona
Panel de administración	No trae. Hubo que construir el panel	Django Admin funciona casi sin código
Curva de entrada	Baja si se conocen los tipos de Python	Más alta: hay que aprender la convención completa
Peso del proyecto	Ligero. Se instala solo lo que se usa	Trae mucho de fábrica, útil o no

11.1 Por qué FastAPI en este proyecto

Pesaron tres razones. La primera, que la documentación automática es parte del entregable: con FastAPI, `/docs` refleja el código sin mantenimiento aparte, y eso evita el problema clásico de una documentación que envejece mal. La segunda, que el Frontend es React y consume JSON: el panel de administración de Django, que es su mayor ventaja, no habría aportado nada porque el panel se construyó a medida. La tercera, que Pydantic permitió que la validación y el contrato de la API fueran la misma declaración, en lugar de dos piezas que hay que mantener sincronizadas.

La contrapartida es honesta: Django REST habría dado gratis el panel de administración y un sistema de usuarios y permisos ya resuelto. Aquí ambos hubo que construirlos, y eso fue trabajo real. A cambio se ganó control sobre el modelo de datos y sobre la forma de cada respuesta.

12. Conclusiones y recomendaciones

12.1 Conclusiones

El proyecto cumplió su objetivo: hay una aplicación en línea, con datos reales, que un tercero puede abrir y usar. Más allá del resultado, quedan varios aprendizajes concretos.

- Las reglas de negocio pertenecen al servidor. Quitar un botón de la pantalla no impide nada: mientras el endpoint siguiera abierto, cualquiera con una sesión podía saltarse la regla. Restringir el registro de ventas a dos roles fue lo que realmente hizo cumplir la política de vender solo por WhatsApp.
- Los documentos emitidos deben guardarse por copia, no por referencia. Que la factura conserve sus propias líneas y los datos del cliente es la diferencia entre un comprobante y una consulta que puede cambiar debajo.
- Probar no es opinar. Varios fallos que parecían resueltos solo aparecieron al ejecutar: una colección de Postman con identificadores fijos que se rompía en la segunda corrida, un ensayo del arranque en modo producción que reveló un error de configuración antes del despliegue, y un desfase de horas entre el reloj de la base y el de los informes que hacía desaparecer del reporte las ventas de la tarde.
- Los detalles del navegador importan. Un modal que no aparecía se debía a un transform residual de una animación en un elemento antecesor, que cambia el punto de referencia de position: fixed. Se resolvió dibujando el modal con un portal.
- La seguridad se juega en lo que no se dice. El ingreso en dos pasos no valida el correo contra el servidor, y la recuperación de contraseña responde lo mismo exista o no la cuenta, para no revelar qué direcciones están registradas.

12.2 Recomendaciones para una versión futura

- Integrar la API de WhatsApp Business para que el pedido enviado quede registrado como solicitud formal y el paso a venta sea trazable, en lugar de depender de que alguien lo transcriba.
- Convertir a asíncronos los endpoints que esperan por entrada y salida, en particular la llamada al proveedor de IA y la generación de documentos, y mover el envío de correo a una tarea en segundo plano para que la petición no espere por el servidor SMTP.
- Añadir paginación con cursor al historial de ventas: la paginación por desplazamiento se degrada cuando la tabla crece.
- Incorporar facturación electrónica ante la DIAN si el negocio lo requiere formalmente.
- Sustituir el plan gratuito del Backend por uno que no suspenda el servicio por inactividad, ya que la primera visita después de un rato tarda cerca de un minuto en responder.
- Guardar un histórico de precios del catálogo para poder analizar márgenes a lo largo del tiempo.

13. Anexos

13.1 Enlaces del proyecto

Recurso	Dirección
Repositorio	https://github.com/tomascardonahincapie/motoshub-react-fastapi
Aplicación publicada	https://motoshub-react-fastapi.vercel.app
API publicada	https://motoshub-react-fastapi.onrender.com
Documentación de la API	https://motoshub-react-fastapi.onrender.com/docs
Estado del servicio	https://motoshub-react-fastapi.onrender.com/salud

13.2 Usuarios de prueba

Las contraseñas de estas cuentas son de demostración y están documentadas en el esquema de la base de datos.

Rol	Correo	Contraseña
Administrador	admin@jhmtech.com	Admin1234
Empleado	empleado@jhmtech.com	Empleado123
Cliente	cliente@jhmtech.com	Cliente123

13.3 Estructura del repositorio

motoshub-react-fastapi/	
backend/	
app/	routers, schemas, crud, models, core, servicios
database/	schema.sql y migraciones
scripts/	carga del esquema, datos de demostracion y verificacion
tests/	137 pruebas automaticas
postman/	coleccion con 86 peticiones
frontend/	
src/	pages, components, context, utils
public/	imagenes del catalogo
evidencias/	capturas y lista de chequeo
DESPLIEGUE.md	paso a paso del despliegue
README.md	descripcion general y puesta en marcha

13.4 Documentos adicionales

- README.md: descripción general, puesta en marcha y verificación del proyecto.
- DESPLIEGUE.md: procedimiento completo de despliegue en Aiven, Render y Vercel, con los errores frecuentes y su solución.
- evidencias/quinto_avance/: capturas del sistema en funcionamiento y lista de chequeo diligenciada.
- backend/.env.example: plantilla de todas las variables de entorno, sin valores reales.